

twyla

A static-site generator whose only authoring language is typst.

Twyla embeds the typst compiler as a Rust library (the typst crate), not the CLI, so it can: route typst files to URLs, intercept asset resolution to fingerprint/optimize images, walk the compiled document for metadata, emit RSS/Atom, and (eventually) do incremental rebuilds.

The first concrete goal is to port samsartor.com (currently zola + tera + markdown, source at ~/Src/site) to twyla + typst, with HTML AST equivalence as the porting harness.

Why pure typst (no template layer)

Considered: a tera-like template engine layered on top of typst, by analogy with zola layering tera on top of markdown. Rejected — typst already provides the moving parts a template language would re-invent:

- **Inheritance** — `base.html + {% block %}` becomes a typst function that takes a body parameter, with show/set rules supplying the defaults.
- **Filters** — typst functions compose better than pipeline syntax.
- **Shortcodes** — already just typst functions, no separate registry.
- **HTML output** — `html.elem(..)` since typst 0.13 lets us emit arbitrary tag + class + attribute structures, which is what tera shortcodes do today.

Cost of **adding** a template language: two scoping models, two escape-hatch conventions (`| safe` vs typst's `html.frame`), and two contexts for the author to switch between.

Cost of **not** adding one: typst's HTML export is younger than tera, so we may hit ergonomic rough edges. That's where twyla earns its keep — wrapping the crate gives us room to shape the output rather than living with the default.

Open questions

1. **Routing model.** Single typst World, with custom content nodes (working names: `asset`, `resource`, `page`) that auto-split into `html` / `svg` / `pdf` outputs at a derived-or-declared path. Multi-entripoint sugar on top: dropping a new `.typ` into `posts/` creates a post without further config. Open: does the path come from the filesystem (mirror layout), from a `metadata()` call inside the doc, or both? Leaning filesystem-by-default, metadata-override.
2. **HTML AST equivalence.** “Match the zola output up to whitespace” needs a concrete differ — parse both sides (`html5ever` / `scraper`), normalize whitespace + attribute order, compare trees, report structural drift. This is the porting feedback loop; without it, every page becomes manual eyeballing. Must be early infrastructure, not a side tool.
3. **Asset pipeline.** Image references inside typst need to be fingerprinted, optimized, and emitted at stable URLs. Sass and JS bundling — probably shell out to existing tools (current site uses vite) rather than reimplement. Open: does twyla own the dev server, or do we keep vite dev alongside?
4. **Metadata extraction.** Title, description, date, draft, custom `extra.*` fields all live in TOML frontmatter today. In typst they become document metadata + `metadata()` calls, queried after compile. Need to verify what `typst::compile` returns and what's queryable from outside the document.
5. **Compile interface.** One World with many entry points (fast, deduplicates imports, manual source-file lifetime management) vs `typst::compile` per page (simple, slow, no cross-page reuse). Leaning single World.

Milestones

1. **HTML-export prototype.** Pin a typst version. Write three throwaway documents that exercise: heading + paragraph, `<div class="...">` with arbitrary class control, inline raw SVG. Verify

each survives HTML export cleanly. If class control or raw-SVG injection is weak, the design changes — find out now.

2. **AST-diff harness.** Standalone Rust binary that takes two HTML files and reports structural equivalence (whitespace + attribute order normalized). Reused throughout porting; written once.
3. **Port one page end-to-end.** Target: `guis-2`. Exercises TOML frontmatter, code blocks with syntax highlighting, tables with embedded shortcodes in cells, horizontal rules, and the centered / `svg` / `diagram` / `image` shortcodes. If `guis-2` round-trips, most of the site will.
4. **Generalize.** Only after `guis-2` matches: routing convention, asset collection, feed generation, dev server.

Findings — experiment 1 (2026-05-17)

Throwaway harness at `src/main.rs` + `experiment.typ`, 115 lines of Rust total. Implements the minimal World, compiles with `typst 0.14.2` + `html` feature, dumps document info, metadata, asset requests, and rendered HTML.

Confirmed:

- **Class control is exact.** `#html.div(class: "container --large")` emits `<div class="container --large">` verbatim. CSS-class-driven porting from zola is unblocked.
- **Document metadata round-trips.** `#set document(title:, author:)` lands in `HtmlDocument::info.title` / `.author` as expected.
- **metadata(..) round-trips with full structured payloads.** Dicts, datetimes, arrays all survive. Query selector: `Selector::Elem(MetadataElem::ELEM, None)`; unpack via `Content::to_packed::<MetadataElem>()` to read the value field. Label-based lookup also works. **Note:** `Selector::can::<MetadataElem>()` returns 0 hits — wrong matcher despite compiling; use `Selector::Elem` instead.
- **HTML escaping is automatic.** A literal `<` inside `#raw(..)` came out as `<code><</code>`.
- **World::file is the asset hook.** `image("teaser.svg")` triggered exactly one call with `path = "teaser.svg"`. The `FileId` carries `vpath` (the rootless requested path) — enough to drive a content-addressed asset pipeline.
- **Library construction gotcha.** HTML export is gated on `Library::builder().with_features([Feature::Html])`. Not a Cargo feature. Without it, compile errors with “html export is only available when `--features html` is passed” (misleading — there is no such Cargo flag). Always emits a warning even when enabled; that’s fine.

Surprise — and a design fork: `image("teaser.svg")` in HTML mode does **not** emit ``. Typst reads the bytes via our World, then base64-encodes them into a `` data URL inline in the HTML. Twyla cannot just intercept `image()` and rewrite the URL — typst owns the rendering. Three viable paths:

1. Define a twyla-provided `asset("foo.svg")` function in the library that **does not** call `image()` — instead, it returns content built from `html.img(src: <our-derived-url>)`. We do asset interning at the function-call site, emit external `<img>` tags, and never use typst’s `image()` for HTML output. **Probably the right answer** — gives us explicit control of class names, `srcset`, lazy-loading, captions, etc.
2. Post-process the HTML: walk the DOM, extract every data: URL, hash the bytes, write them to disk, rewrite the `src` to the new URL. Works but feels brittle — any future typst optimization to elide tiny images or fold multiple references would silently change our output.
3. Let typst inline assets and accept the data-URL bloat. Fine for tiny decorative SVGs, terrible for the site’s 50KB SVG diagrams. Not serious for our content.

Lean: option 1. The asset/resource/page custom-content-type idea from the earlier sketch maps cleanly here — `asset()` is one of those types, and it bypasses `image()` entirely.

Other observations:

- Typst emits a full HTML5 document by default (doctype, `<html>`, `<head>` with meta charset, viewport meta, `<title>`, author meta). For matching the zola output we'll need to either suppress this and emit our own `<head>` shell, or replace the auto-emitted `<head>` contents via... some mechanism not yet discovered.
- Output is pretty-printed with 2-space indentation. The AST-diff harness can ignore this trivially but worth knowing.
- `typst-kit's Fonts::searcher().include_system_fonts(true).search()` is one line and gave us system fonts with zero ceremony — keep using it during development.

Findings — experiment 2 (2026-05-17)

Repointed at `typst main` (commit `de6f400`, 2026-04-11) for the bundle export feature merged in 7964 Plan: switch back to `crates.io 0.15.x` once it ships.

Three `#document(path, title: ..)` calls + one `#asset(path, bytes)` in `experiment.typ`. Compiled as `typst::compile::<Bundle>(&world)` with `Library::builder().with_features([Feature::Html, Feature::Bundle])`.

Worked first try (after API breakage cleanup — see below):

- **Four files emitted from one compile:** `index.html`, `blog/index.html`, `blog/guis-2.html`, `style.css`. The router question is **closed**: `#document(path, ..)` is the primitive. We don't need filesystem-mirror conventions or manifest files; we can build sugar on top later.
- **Cross-document linking is automatic and path-correct.** Labels in one doc, referenced from another:
 - ▶ `index.html` → `<a href="blog/index.html#blog">`
 - ▶ `blog/index.html` → `<a href="../index.html#home-meta">` (*uses ...!*)
 - ▶ `blog/index.html` → `<a href="guis-2.html#post-1">` (*sibling*)

Typst computes relative paths. Zero URL construction work for us.

- **Labels auto-emit as HTML id anchors.** `<my-label>` placed in content becomes `<span id="my-label"></span>` inline. Good enough for `#link()` to work; placement is up to the author (typst attaches to the closest preceding element).
- **Per-document metadata isolation works.** `doc.introspector().query(..)` on each `BundleDocument::Html` returns only that doc's metadata.
- **Bundle-wide metadata also works.** `bundle.introspector.query(..)` returns metadata from across all docs. This is the **primitive for feed generation**: query all kind: "post" entries → emit the feed.
- **Assets are direct.** `#asset("style.css", bytes("body { ... }"))` produces a `BundleFile::Asset(Bytes)` at the requested path. For optimized images we'll preprocess bytes in Rust, then hand them to typst via `#asset`.

Surprises / notes:

- **Smart quotes default on:** "Aren't" rendered as "Aren't" (U+2019). Disable per-document with `#set smartquote(enabled: false)` if matching Zola output requires it.
- **Heading level offset:** = Heading emits `<h2>`, not `<h1>` — typst reserves `<h1>` for the document title (configurable via `html.frame` rules; revisit during porting).
- `<html lang="en">` is **auto-set** on main (was missing in 0.14.2). Good default.
- **Only warning:** "bundle export is experimental." Same status as HTML export — fine for our purposes.
- **Asset hook (`World::file`) was not called** this experiment because we used `bytes("...")` (literal) for the CSS. Already verified in experiment 1 that `image("foo")` triggers it.

Architectural decisions confirmed:

1. Router primitive: `#document(path, ..)` (typst-owned, not twyla-owned).
2. Cross-doc links: typst-owned.
3. Feed generation: written as `#document("atom.xml", ...)` in twyla's library, querying `bundle.introspector` for posts.
4. Multi-entypoint sugar: twyla scans `content/*.typ`, generates a `main.typ` in memory that emits one `#document(..)` per file, compiles the bundle.
5. Asset optimization: still twyla's job. Either (a) Rust scans references and preprocesses before exposing via `World::file`, or (b) custom typst functions in twyla's library (e.g. `optimized-image`) that do the work and emit `html.img(src: <hashed-url>) + #asset`.

Breaking changes between 0.14.2 and main (for future-us pinning bumps):

- `typst-kit`: feature fonts → split into `scan-fonts` + `embedded-fonts`. API: `Fonts::searcher()` → gone. New: `FontStore::new()` + `extend(typst_kit::fonts::embedded())` + `extend(typst_kit::fonts::system())`.
- `FileId::new(None, vpath)` → `FileId::new(RootedPath::new(VirtualRoot::Project, vpath))`.
- `VirtualPath::new(s)` now returns `Result<Self, PathError>`.
- `VirtualPath::as_rootless_path()` deprecated → `get_without_slash()` (returns `&str`, not `&Path`).
- `HtmlDocument::info`, `introspector` fields → private; use methods `.info()` and `.introspector()`. Requires use `typst_library::model::Document`.
- `World::today(_offset: Option<i64>)` → `Option<typst::foundations::Duration>`.
- `bundle.introspector` is `Arc<BundleIntrospector>`; `.query()` requires use `typst::introspection::Introspector`; in scope.
- New crate: `typst-bundle` (pinned same rev).
- New `Feature::Bundle` enum variant alongside `Feature::Html`.

Fork vs library: tracked decisions

Standing assumption: **do not fork typst**. Build twyla as a Rust binary that consumes typst as a library, supplies a custom `World`, ships a small typst library of helper functions, and post-processes the bundle. If something is genuinely better as an upstream change, **upstream it** before forking. The fork threshold should be “library + upstreaming both proven inadequate.”

Why this default: typst's HTML/bundle work is actively evolving in main. A fork puts us permanently downstream of grammar, library, and HTML-output changes — that's a real maintenance tax, paid forever, for benefits that are mostly reachable from outside.

Per-feature analysis below. If we ever hit a real wall, **update this table** with what we tried and why it didn't work — that's the evidence base for reconsidering.

Feature	Upstream-friendly path	Fork would buy us	Verdict
Asset fingerprinting (e.g. <code>style.abc.css</code> + ref rewriting)	Post-export pass over bundle: walk <code>VirtualFs</code> , hash bytes, rewrite paths in both HTML strings and asset names. Use <code>html5ever</code> for parsing.	Cleaner integration with introspector for ref resolution.	Library
Image optimization (AVIF/WebP, <code>srcset</code> , responsive sizes)	Twyla-provided <code>image(..)</code> function that bypasses typst's <code>image()</code> , preprocesses bytes, emits	Could replace typst's default <code>image()</code> behavior in HTML mode (currently data-URL inlines).	Library + maybe upstream

Feature	Upstream-friendly path	Fork would buy us	Verdict
	<code>html.img(srcset: ..) + #asset(..).</code>		
Pretty URLs (<code>/post/foo/</code> not <code>/post/foo.html</code>)	Emit <code>#document("post/foo/index.html", ..)</code> in generated <code>main.typ</code> .	Nothing — equivalent.	Library
Path computation / lang prefixes / trailing-slash policy	All happens in our generated <code>main.typ</code> before compile.	Nothing — equivalent.	Library
HTML syntax mode (JSX-style <code><div>..<div>< code="">)</div><></code>	<code>html.div(class: "..") [..]</code> is already fine.	Minor ergonomic win, large maintenance cost (parser + AST + downstream of all grammar evolution).	No fork
Hot reload / live preview	Inject WebSocket script into HTML during dev-server post-processing.	Nothing — wrong layer.	Library
RSS/Atom/sitemap generation	<code>#document("atom.xml", ..)</code> querying <code>bundle.introspector</code> for posts.	Nothing — wrong layer.	Library
Per-document smartquote / language / heading-offset control	Already configurable via <code>#set smartquote(..)</code> etc per document.	Possibly: nicer defaults for HTML mode. Worth upstreaming.	Library or upstream
Incremental compilation across watch cycles	Hold one world across runs; lean on comemo memoization.	Finer-grained invalidation hooks; persistent on-disk cache.	Library; revisit if cold-start becomes a perf wall
Reading file mtime / external metadata inside <code>typst</code>	Supply data via <code>sys.inputs</code> from Rust-side directory scan.	Direct filesystem stat exposure. Probably bad for sandboxing.	Library
Async / HTTP during compile (OpenGraph fetches, etc.)	Do it in Rust pre-compile, inject results via <code>sys.inputs</code> .	Async <code>typst</code> — large architectural change, probably never wanted.	No fork

Things we should at least keep an eye on for upstreaming:

1. Optional hook for `image()` to emit external `src` instead of data URLs in HTML mode — would benefit anyone doing HTML export at scale.
2. Per-document defaults override (e.g. default `smartquote(enabled: false)` for a specific bundle target).
3. Introspector exposure of cross-document link resolution data, useful for asset-fingerprinting passes.

If we open issues for these, link them here so the next person reading this section can see whether the calculus has changed.

Notes

- VCS: both `~/Src/twyla` and `~/Src/site` are managed with `jj`. **Rule:** always `jj new` **before** editing — `jj` has no staging area, so edits land in whatever revision the working copy points at.

Recovering from an accidental commit requires `jj split`, which is interactive and cannot be driven from a non-interactive shell.

- Typst version: TBD (pin when starting milestone 1).
- Tera shortcodes in scope for porting: `centered`, `diagram`, `image`, `math`, `svg`, `tagline`, `var` (`~/Src/site/templates/shortcodes/`).
- Tera templates in scope: `base.html`, `page.html`, `section.html`, `index.html`, `components.html`, `404.html`, `atom.xml`, `paperpillls.html`.
- Known porting hazards: ROT13-encoded email obfuscation in footer, conditional asset loading (`page.extra.tilings`), SVG inlining with font-family rewriting.